

Abstract

In today's digital environment, network security has become a critical concern due to the increasing number of cyber threats such as unauthorized access, port scanning, and network exploitation. Intrusion Detection Systems (IDS) play a vital role in identifying suspicious activities within a network. However, traditional IDS solutions primarily detect attacks based on IP addresses and fail to associate them with actual users or identities.

This project presents the implementation of a real-time Intrusion Detection System using Snort 3, integrated with a basic Identity and Access Management (IAM) module. The system monitors live network traffic, detects malicious activities such as ICMP ping attacks and port scans, and generates alerts. These alerts are further processed using a Python script that maps IP addresses to specific user identities. Based on detection, automated actions such as blocking malicious users using firewall rules (iptables) are performed.

The system is implemented in a Kali Linux environment and tested using simulated attacks via Nmap. The results demonstrate that the integration of IDS with IAM enhances security by not only detecting threats but also identifying and responding to attackers effectively.

Objectives

The primary objectives of this project are:

1. To design and implement an Intrusion Detection System using Snort 3
2. To monitor network traffic in real-time
3. To detect malicious activities such as port scanning and ICMP attacks
4. To map attacker IP addresses to user identities
5. To automate responses such as blocking attackers
6. To integrate IDS with Identity and Access Management concepts
7. To build a practical and deployable cybersecurity solution

Problem Statement

Traditional IDS systems have several limitations:

- They only display IP addresses of attackers
- They do not identify the actual user behind an attack
- They lack automated response mechanisms
- They require manual analysis and intervention

Because of these limitations, security teams often struggle to respond quickly and effectively.

This project addresses these issues by:

- Mapping IP addresses to user identities
- Automatically detecting attacks
- Blocking malicious users in real time
- Enhancing IDS with IAM capabilities

Theory

Intrusion Detection System (IDS)

An Intrusion Detection System is a security tool used to monitor network traffic and detect suspicious activities. IDS can be classified into:

Network-based IDS (NIDS) – monitors entire network traffic

Host-based IDS (HIDS) – monitors a single system

In this project, we use a **Network-based IDS**.

Snort 3

Snort is an open-source IDS developed for real-time traffic analysis and packet logging. It works by:

1. Capturing packets from the network
2. Comparing them with predefined rules
3. Generating alerts when matches are found

Snort 3 is the latest version with improved modular architecture and performance.

Identity and Access Management (IAM)

IAM ensures that the right users have appropriate access to resources. In this project:

- IP address → User mapping is used
- Attacks are linked to identities
- Actions are taken based on identity

Nmap (Network Mapper)

- Nmap is a tool used for:
- Port scanning
- Network discovery
- Vulnerability assessment

It is used here to simulate attacks.

iptables

iptables is a Linux firewall utility used to:

- Allow or block traffic
- Filter packets
- Protect systems from attacks

WorkFlow

1. Attacker machine sends malicious traffic
2. Snort captures and analyzes packets
3. Snort detects suspicious activity
4. Alerts are generated and stored
5. Python script reads alert file
6. IP is mapped to user identity
7. System generates identity-based alert
8. Firewall blocks attacker using iptables

System Requirements

Hardware Requirements

- Minimum 2 machines (or one system with network simulation)
- Processor: Intel i3 or higher
- RAM: 4GB or more

Software Requirements

- Kali Linux / Ubuntu
- Snort 3
- Python 3
- Nmap
- Terminal (CLI environment)

Implementation Steps

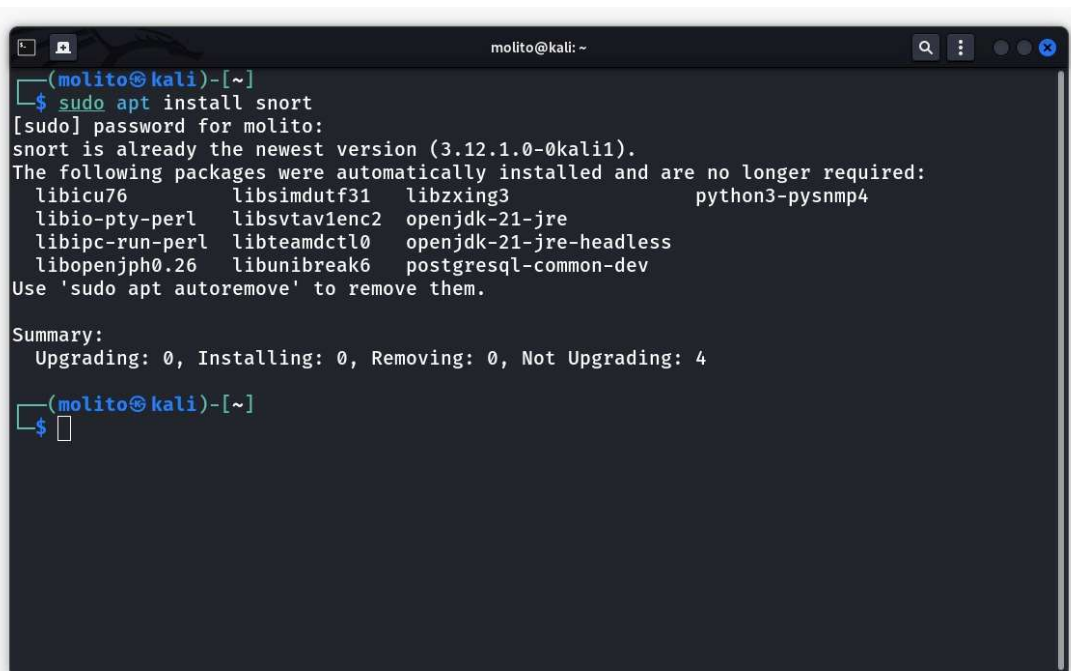
Step 1: Installing Snort

Command used:

```
sudo apt install snort
```

Explanation:

- sudo → runs command with administrative privileges
- apt install → installs software packages
- snort → IDS tool



```
(molito@kali)-[~]
$ sudo apt install snort
[sudo] password for molito:
snort is already the newest version (3.12.1.0-0kali1).
The following packages were automatically installed and are no longer required:
 libicu76      libsimdutf31  libzxing3      python3-pysnmp4
 libio-pty-perl  libsvtav1enc2  openjdk-21-jre
 libipc-run-perl  libteamctl0   openjdk-21-jre-headless
 libopenjph0.26  libunibreak6  postgresql-common-dev
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 4
(molito@kali)-[~]
$
```

Step 2: Checking Network Configuration

Command:

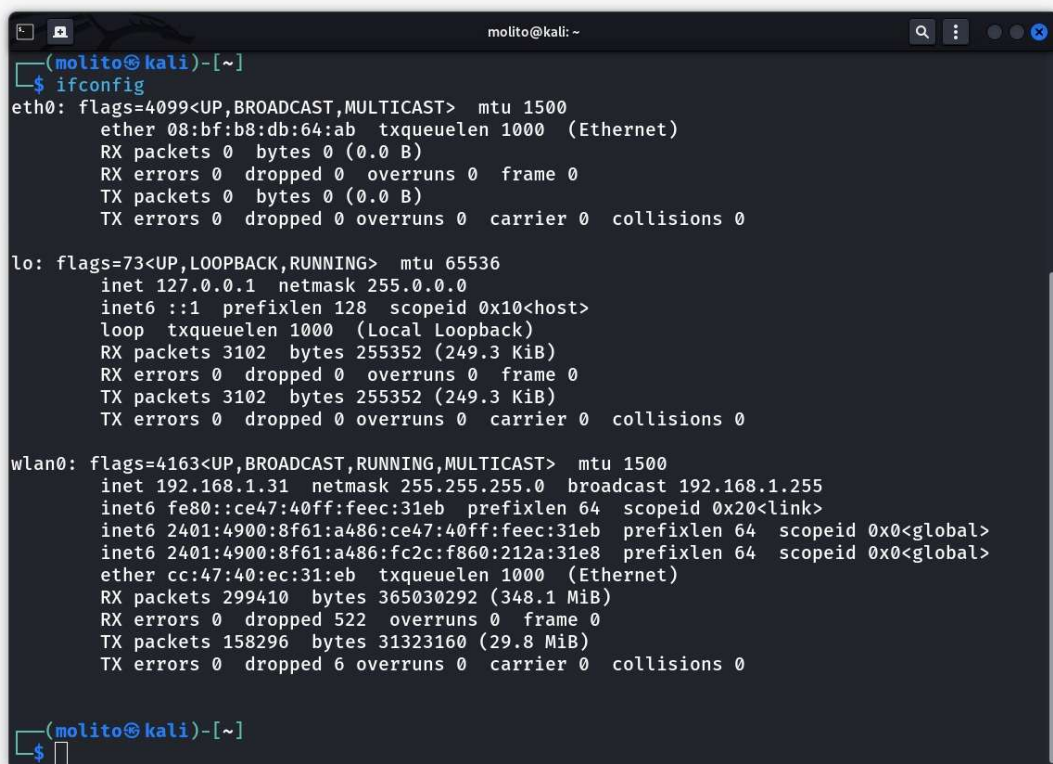
```
ifconfig
```

Explanation:

- Displays network interfaces
- Shows IP address, MAC address, packets

output:

- Interface: wlan0
- IP Address: 192.168.1.31



```
(molito@kali)-[~]
$ ifconfig
eth0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 08:bf:b8:db:64:ab txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 3102 bytes 255352 (249.3 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 3102 bytes 255352 (249.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlan0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.31 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::ce47:40ff:feec:31eb prefixlen 64 scopeid 0x20<link>
    inet6 2401:4900:8f61:a486:ce47:40ff:feec:31eb prefixlen 64 scopeid 0x0<global>
    inet6 2401:4900:8f61:a486:fc2c:f860:212a:31e8 prefixlen 64 scopeid 0x0<global>
    ether cc:47:40:ec:31:eb txqueuelen 1000 (Ethernet)
    RX packets 299410 bytes 365030292 (348.1 MiB)
    RX errors 0 dropped 522 overruns 0 frame 0
    TX packets 158296 bytes 31323160 (29.8 MiB)
    TX errors 0 dropped 6 overruns 0 carrier 0 collisions 0

(molito@kali)-[~]
$
```

Step 3: Configuring Snort

File edit:

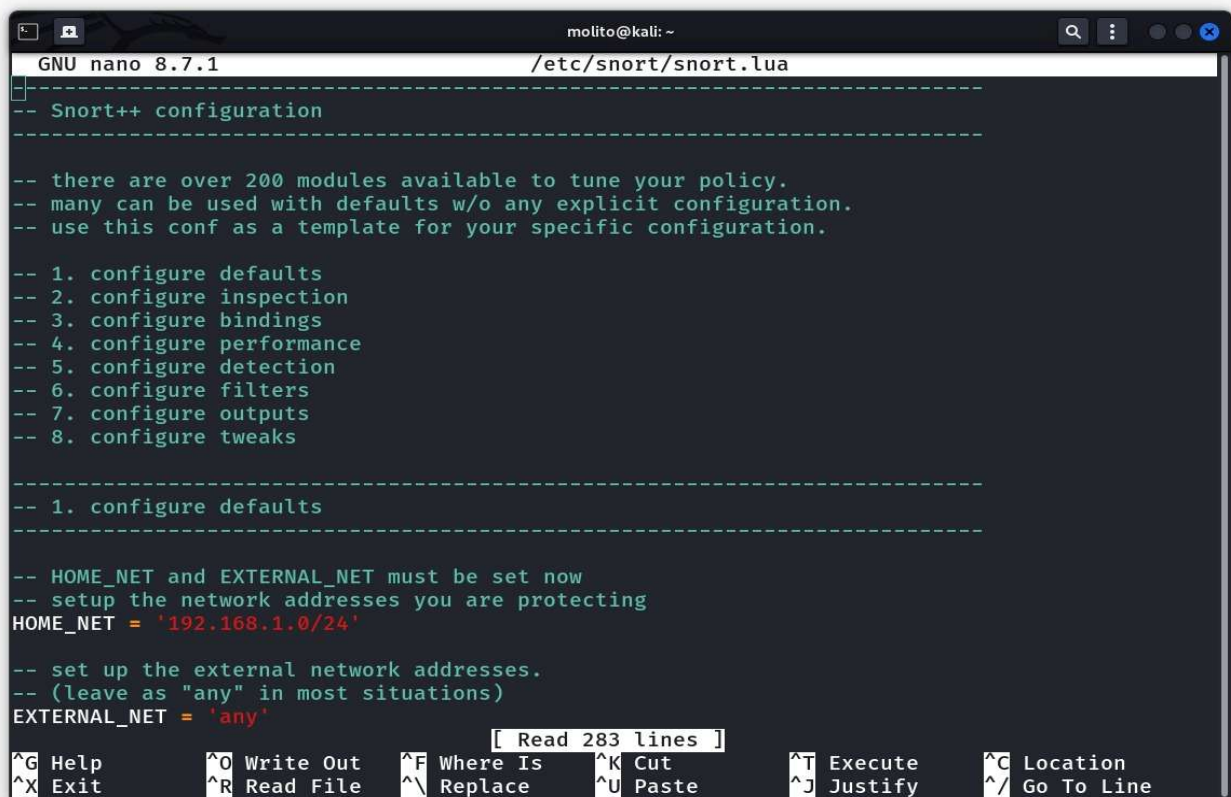
```
sudo nano /etc/snort/snort.lua
```

set:

```
HOME_NET = '192.168.1.0/24'  
EXTERNAL_NET = 'any'
```

Explanation:

- HOME_NET → defines protected network
- /24 → subnet range (192.168.1.0–255)
- EXTERNAL_NET → outside traffic



The screenshot shows a terminal window with the nano text editor open at the file `/etc/snort/snort.lua`. The editor displays the Snort++ configuration file. The configuration includes a list of modules to tune, a list of configuration steps (1-8), and the specific settings for `HOME_NET` and `EXTERNAL_NET`. The `HOME_NET` is set to `'192.168.1.0/24'` and `EXTERNAL_NET` is set to `'any'`. The terminal window title is `molito@kali: ~`. The nano editor status bar at the bottom shows `[ Read 283 lines ]` and various keyboard shortcuts.

```
GNU nano 8.7.1 /etc/snort/snort.lua  
-----  
-- Snort++ configuration  
-----  
  
-- there are over 200 modules available to tune your policy.  
-- many can be used with defaults w/o any explicit configuration.  
-- use this conf as a template for your specific configuration.  
  
-- 1. configure defaults  
-- 2. configure inspection  
-- 3. configure bindings  
-- 4. configure performance  
-- 5. configure detection  
-- 6. configure filters  
-- 7. configure outputs  
-- 8. configure tweaks  
  
-----  
-- 1. configure defaults  
-----  
  
-- HOME_NET and EXTERNAL_NET must be set now  
-- setup the network addresses you are protecting  
HOME_NET = '192.168.1.0/24'  
  
-- set up the external network addresses.  
-- (leave as "any" in most situations)  
EXTERNAL_NET = 'any'  
  
[ Read 283 lines ]  
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location  
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

Step 4: Running Snort

Command:

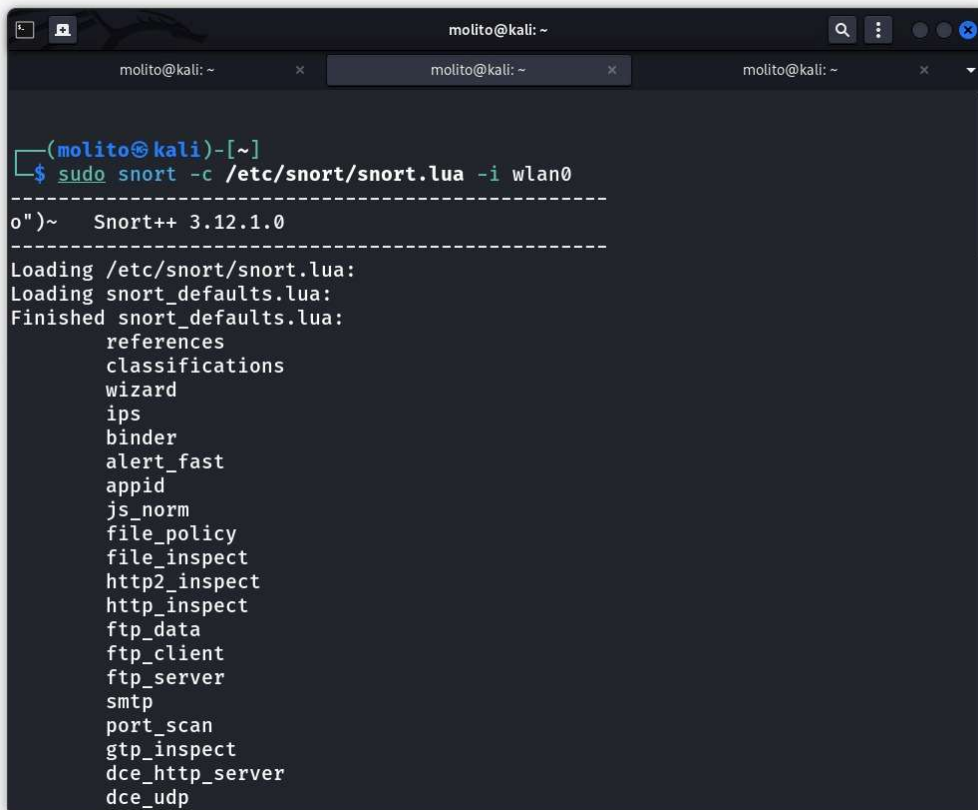
```
sudo snort -c /etc/snort/snort.lua -i wlan0
```

Explanation:

- -c → configuration file
- -i wlan0 → network interface

Output :

- Rules loaded: 869
- Snort running successfully
- Packet processing started

A terminal window screenshot showing the execution of the Snort command. The prompt is (molito@kali)-[~]. The command entered is \$ sudo snort -c /etc/snort/snort.lua -i wlan0. The output shows the Snort++ version 3.12.1.0, followed by the loading of /etc/snort/snort.lua and snort_defaults.lua. A list of loaded rules is displayed, including references, classifications, wizard, ips, binder, alert_fast, appid, js_norm, file_policy, file_inspect, http2_inspect, http_inspect, ftp_data, ftp_client, ftp_server, smtp, port_scan, gtp_inspect, dce_http_server, and dce_udp.

```
(molito@kali)-[~]  
$ sudo snort -c /etc/snort/snort.lua -i wlan0  
-----  
o")~  Snort++ 3.12.1.0  
-----  
Loading /etc/snort/snort.lua:  
Loading snort_defaults.lua:  
Finished snort_defaults.lua:  
  references  
  classifications  
  wizard  
  ips  
  binder  
  alert_fast  
  appid  
  js_norm  
  file_policy  
  file_inspect  
  http2_inspect  
  http_inspect  
  ftp_data  
  ftp_client  
  ftp_server  
  smtp  
  port_scan  
  gtp_inspect  
  dce_http_server  
  dce_udp
```

```
molito@kali: ~  
Finished /etc/snort/snort.lua:  
Loading file_inspect.rules_file:  
Loading file_magic.rules:  
Finished file_magic.rules:  
Finished file_inspect.rules_file:  
Loading ips.rules:  
Loading /etc/snort/rules/local.rules:  
Finished /etc/snort/rules/local.rules:  
Finished ips.rules:  
-----  
ips policies rule stats  
      id loaded  shared enabled  file  
      1   869      0    869  /etc/snort/snort.lua  
-----  
rule counts  
  total rules loaded: 869  
    text rules: 222  
  builtin rules: 647  
  option chains: 869  
  chain headers: 5  
-----  
port rule counts  
      tcp  udp  icmp  ip  
  any   647   0    1    0  
  dst    2   0    0    0  
  total 649   0    1    0  
-----  
service rule counts      to-srv  to-cli  
      file_id:      219    219  
      total:      219    219
```

```
molito@kali: ~  
dst      2      0      0      0  
total    649     0      1      0  
-----  
service rule counts      to-srv  to-cli  
      file_id:      219    219  
      total:      219    219  
-----  
fast pattern groups  
      to_server: 1  
      to_client: 1  
-----  
search engine (ac_bnfa)  
      instances: 2  
      patterns: 438  
      pattern chars: 2602  
      num states: 1832  
      num match states: 392  
      memory scale: KB  
      total memory: 71.2812  
      pattern memory: 19.6484  
      match list memory: 28.4375  
      transition memory: 22.9453  
appid: MaxRss diff: 2828  
appid: patterns loaded: 300  
-----  
pcap DAQ configured to passive.  
Commencing packet processing  
Retry queue interval is: 200 ms  
++ [0] wlan0  
[ ]
```

Step 5: Simulating Attack using Nmap

Command:

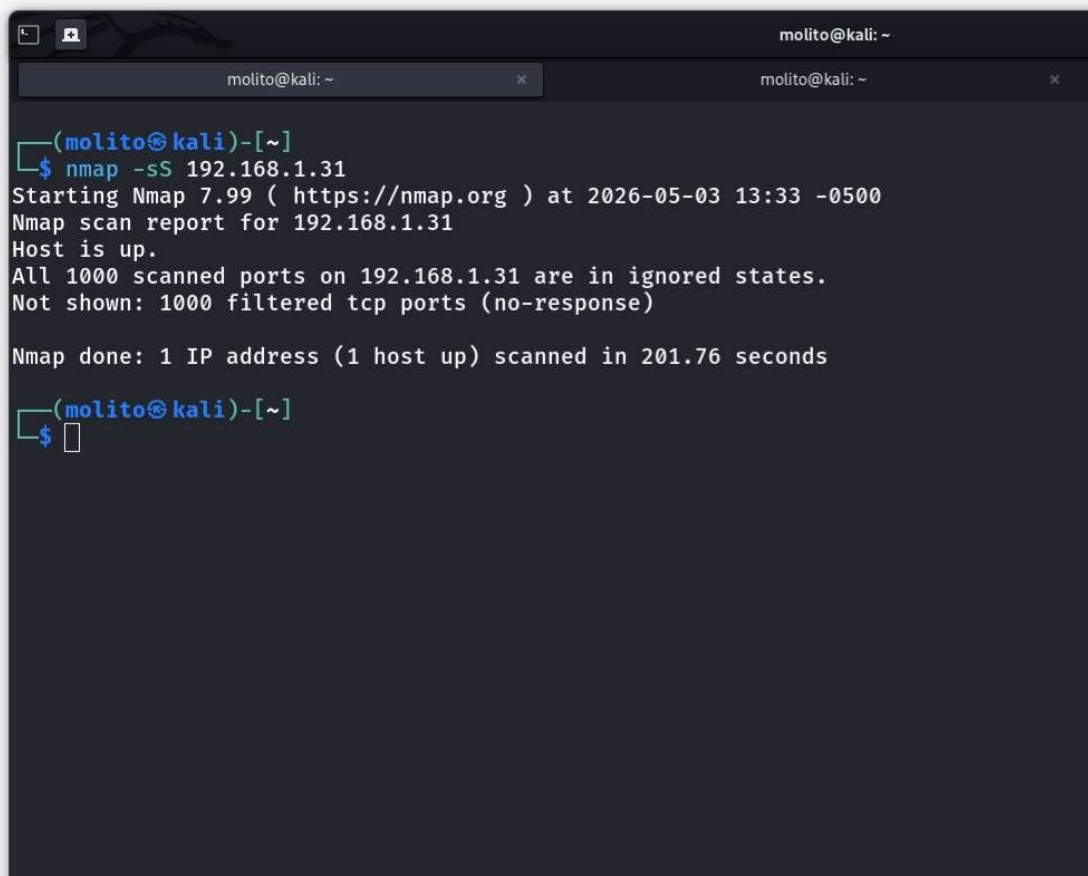
```
nmap -sS 192.168.1.31
```

Explanation:

- -sS → SYN scan (stealth scan)
- Sends packets to detect open ports

Output shows:

- All ports filtered (because blocked later)



```
(molito@kali)-[~]  
$ nmap -sS 192.168.1.31  
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-03 13:33 -0500  
Nmap scan report for 192.168.1.31  
Host is up.  
All 1000 scanned ports on 192.168.1.31 are in ignored states.  
Not shown: 1000 filtered tcp ports (no-response)  
  
Nmap done: 1 IP address (1 host up) scanned in 201.76 seconds  
  
(molito@kali)-[~]  
$
```

Step 6: Viewing Alerts

Command:

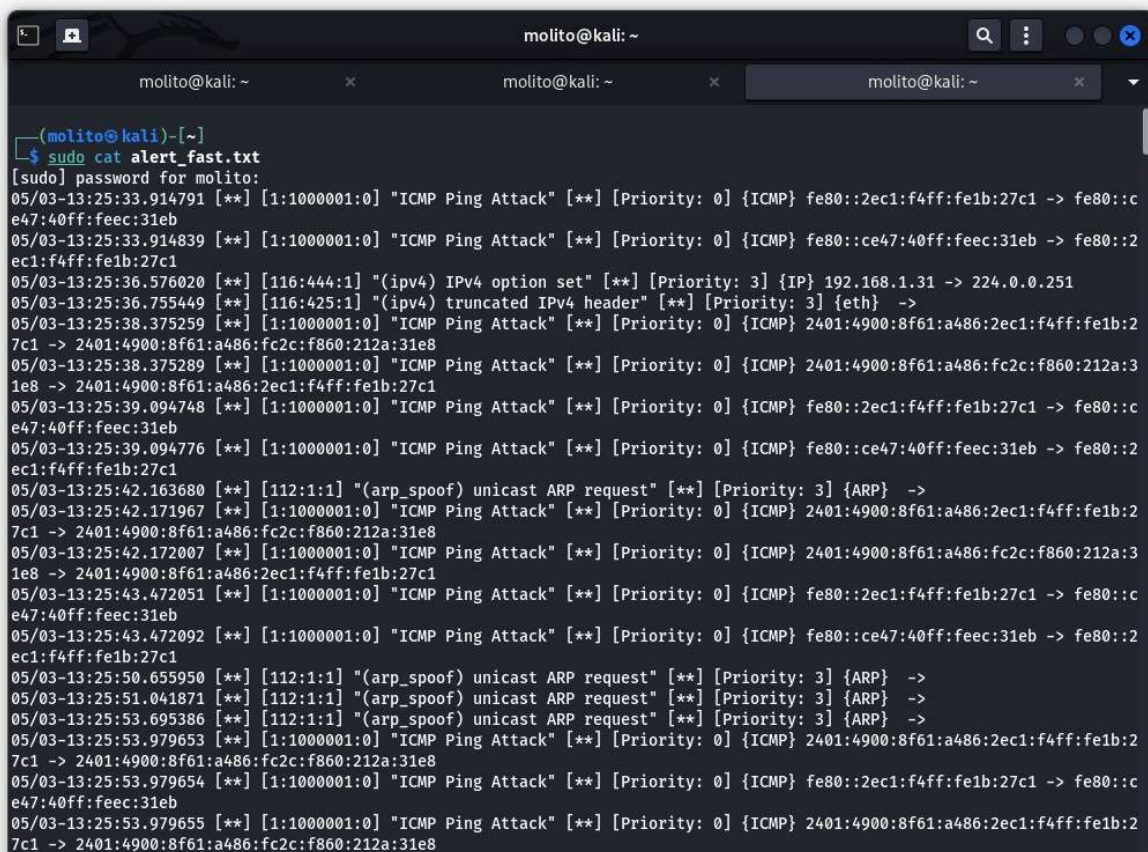
```
cat alert_fast.txt
```

Explanation:

- Displays Snort alerts
- Alerts include:

Examples from output:

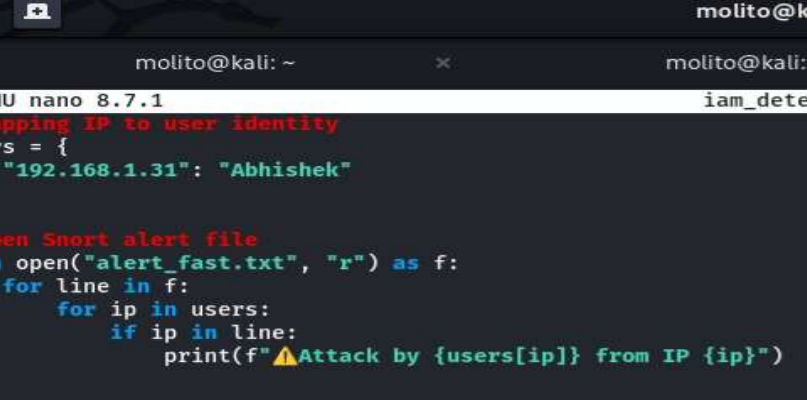
- ICMP Ping Attack
- ARP Spoof detection



```
(molito@kali)-[~]
└─$ sudo cat alert_fast.txt
[sudo] password for molito:
05/03-13:25:33.914791 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} fe80::2ec1:f4ff:fe1b:27c1 -> fe80::c
e47:40ff:feec:31eb
05/03-13:25:33.914839 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} fe80::ce47:40ff:feec:31eb -> fe80::2
ec1:f4ff:fe1b:27c1
05/03-13:25:36.576020 [**] [116:444:1] "(ipv4) IPv4 option set" [**] [Priority: 3] {IP} 192.168.1.31 -> 224.0.0.251
05/03-13:25:36.755449 [**] [116:425:1] "(ipv4) truncated IPv4 header" [**] [Priority: 3] {eth} ->
05/03-13:25:38.375259 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} 2401:4900:8f61:a486:2ec1:f4ff:fe1b:2
7c1 -> 2401:4900:8f61:a486:fc2c:f860:212a:31e8
05/03-13:25:38.375289 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} 2401:4900:8f61:a486:fc2c:f860:212a:3
1e8 -> 2401:4900:8f61:a486:2ec1:f4ff:fe1b:27c1
05/03-13:25:39.094748 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} fe80::2ec1:f4ff:fe1b:27c1 -> fe80::c
e47:40ff:feec:31eb
05/03-13:25:39.094776 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} fe80::ce47:40ff:feec:31eb -> fe80::2
ec1:f4ff:fe1b:27c1
05/03-13:25:42.163680 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/03-13:25:42.171967 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} 2401:4900:8f61:a486:2ec1:f4ff:fe1b:2
7c1 -> 2401:4900:8f61:a486:fc2c:f860:212a:31e8
05/03-13:25:42.172007 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} 2401:4900:8f61:a486:fc2c:f860:212a:3
1e8 -> 2401:4900:8f61:a486:2ec1:f4ff:fe1b:27c1
05/03-13:25:43.472051 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} fe80::2ec1:f4ff:fe1b:27c1 -> fe80::c
e47:40ff:feec:31eb
05/03-13:25:43.472092 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} fe80::ce47:40ff:feec:31eb -> fe80::2
ec1:f4ff:fe1b:27c1
05/03-13:25:50.655950 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/03-13:25:51.041871 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/03-13:25:53.695386 [**] [112:1:1] "(arp_spoof) unicast ARP request" [**] [Priority: 3] {ARP} ->
05/03-13:25:53.979653 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} 2401:4900:8f61:a486:2ec1:f4ff:fe1b:2
7c1 -> 2401:4900:8f61:a486:fc2c:f860:212a:31e8
05/03-13:25:53.979654 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} fe80::2ec1:f4ff:fe1b:27c1 -> fe80::c
e47:40ff:feec:31eb
05/03-13:25:53.979655 [**] [1:1000001:0] "ICMP Ping Attack" [**] [Priority: 0] {ICMP} 2401:4900:8f61:a486:2ec1:f4ff:fe1b:2
7c1 -> 2401:4900:8f61:a486:fc2c:f860:212a:31e8
```

Step 7: Python IAM Integration

Code:



```
GNU nano 8.7.1 iam_detection.py
# Mapping IP to user identity
users = {
    "192.168.1.31": "Abhishek"
}

# Open Snort alert file
with open("alert_fast.txt", "r") as f:
    for line in f:
        for ip in users:
            if ip in line:
                print(f"⚠️Attack by {users[ip]} from IP {ip}")
```

Explanation:

- Dictionary maps IP → User
- File is read line by line
- If IP found → prints user identity

Output shows:

- Attack by Abhishek from IP 192.168.1.31

[illegible]

Step 8: Blocking Attacker

Command:

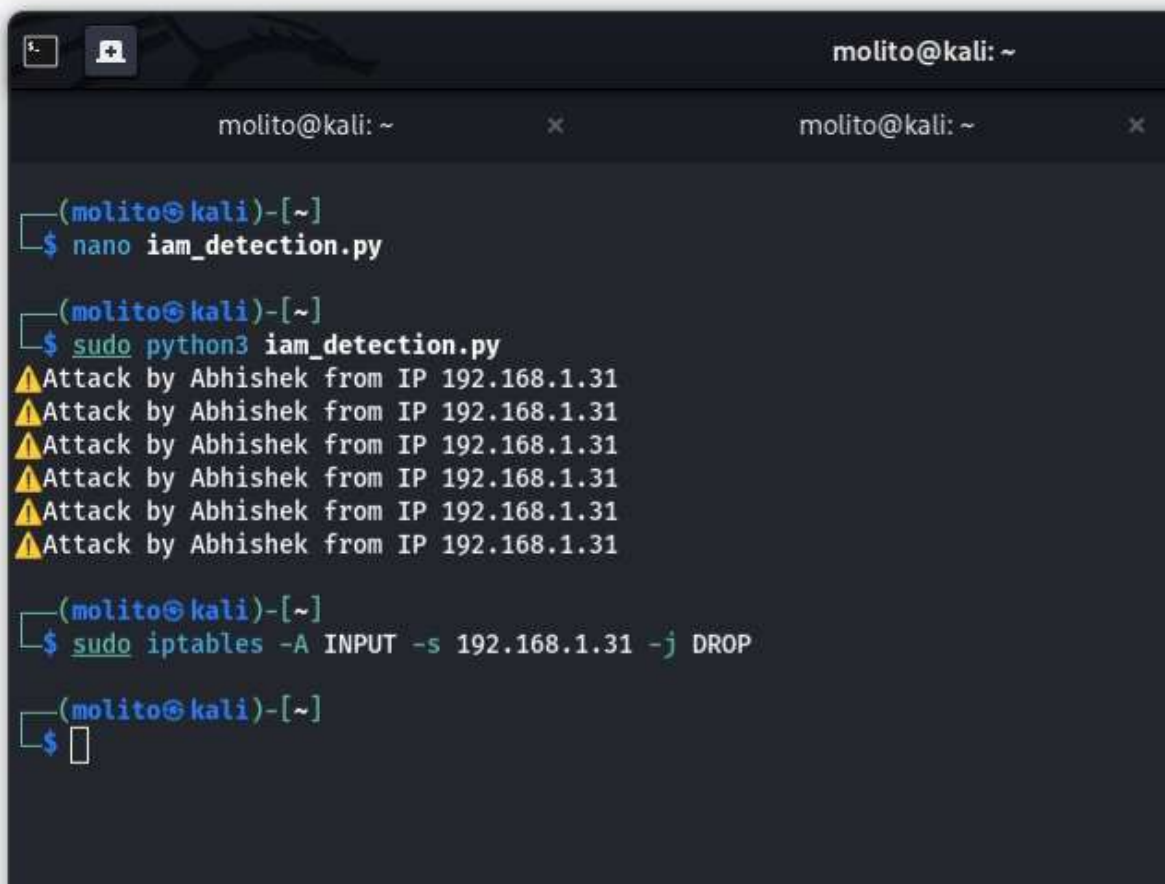
```
sudo iptables -A INPUT -s 192.168.1.31 -j DROP
```

Explanation:

- -A INPUT → append rule
- -s → source IP
- -j DROP → block traffic

Effect:

- All packets from attacker are dropped
- System becomes protected



```
molito@kali: ~  
molito@kali: ~  
molito@kali: ~  
(molito@kali)-[~]  
$ nano iam_detection.py  
(molito@kali)-[~]  
$ sudo python3 iam_detection.py  
⚠️Attack by Abhishek from IP 192.168.1.31  
⚠️Attack by Abhishek from IP 192.168.1.31  
⚠️Attack by Abhishek from IP 192.168.1.31  
⚠️Attack by Abhishek from IP 192.168.1.31  
⚠️Attack by Abhishek from IP 192.168.1.31  
⚠️Attack by Abhishek from IP 192.168.1.31  
(molito@kali)-[~]  
$ sudo iptables -A INPUT -s 192.168.1.31 -j DROP  
(molito@kali)-[~]  
$
```

Results

The system successfully:

- Detected ICMP ping attacks
- Detected ARP spoof attempts
- Generated real-time alerts
- Mapped IP to user identity
- Blocked attacker using firewall
- After blocking:
- Nmap scan showed all ports filtered
- No response received
- This proves the system is working as an Intrusion Prevention System (IPS).

Advantages

- Real-time intrusion detection
- Automatic response system
- Identity-based attack detection
- Lightweight and efficient
- Easy to implement
- Works on local networks

Limitations

- Static IP-to-user mapping
- No graphical interface
- Limited scalability
- Manual rule configuration required
- Works mainly in controlled environments

Conclusion

This project successfully demonstrates the implementation of a real-time Intrusion Detection System using Snort 3 integrated with Identity and Access Management concepts. Unlike traditional IDS systems that only detect attacks, this system enhances security by identifying the user behind the attack and taking automated action.

The integration of Snort, Python, and iptables creates a complete security pipeline that detects, analyzes, and responds to threats efficiently. The project highlights how combining IDS with IAM can significantly improve network security in real-world environments.